

Reflection: Grady Booch's Vision of Software Engineering in the Age of AI

Jeff Wu - ICS 613, Assignment 01 - 2026-06-01

I enjoyed Booch's podcast and lecture on AI and the three golden ages. In my career, I did not experience the first two, but his description of the third rings true to me. Starting around 2000-2001, I experienced web development shifting from building bespoke applications to building within frameworks (Fusebox, Struts, ASP.NET), to using open source platforms (Drupal, WordPress), and finally to SaaS offerings like Salesforce and cloud platforms (AWS, Azure). I agree with Booch about the shifting of dominant problems with this evolution. As composable services eased the management of technical complexity, scaling became the new bottleneck. Once the cloud solved worldwide scaling, harder problems appeared, such as regulatory compliance while operating in other countries, and having the operational manpower needed to support technology at that scale.

AI has changed a lot of the software development practices I know. In the past, if we wanted to discuss a new feature, it would involve meetings and discussions to even decide what direction to go in, because writing code was time-consuming and we wanted to get it right. With AI, we can simply have AI generate PRs to demonstrate our ideas. Rather than endless hours of debate, we compare real options side by side and see the tradeoffs in actual code. This makes planning conversations much more productive.

With AI, we gain a lot in this industry. The act of coding itself is no longer a bottleneck. We can now have high-quality code that used to take days or weeks written in a matter of minutes or hours. Onboarding onto a new team is extremely fast; I was able to create substantial PRs on the first day of onboarding onto a new team. In the past, this would have taken at least one week of ramp-up time before a PR of any substance could be created. On the other hand, I also worry that non-programmers (and perhaps even some programmers) don't sufficiently review and verify the AI output, or worse, don't understand the code that is produced, which can lead to quality issues, increased cyberattack vectors, and many other unwanted outcomes.

I agree with Booch that we will not automate software engineers away entirely, but software engineers will need to adapt to stay relevant. Software engineers will increasingly need to have strong collaboration and planning skills, and learn how to use AI in a deliberate, precise way that builds durable software.

AI is a powerful force multiplier within the software engineering industry, but it can multiply our mistakes if we let it. This is why software engineers must not treat AI as an existential crisis, but rather should embrace it and evolve along with it, just as developers have in the previous golden ages.

AI use disclosure: I wrote this reflection myself and used an AI assistant to proofread. All views and wording are my own.